# Formalisms Every Computer Scientist Should Know: Homework Solutions

## 1 Homework 1

**Theorem 1.1.** *If there is a one-to-one function on a set $A$ to a subset of a set $B$ and there is also a one-to-one function on $B$ to a subset of $A$, then $A$ and $B$ are equipollent.*

*Proof.* We need to show that there exists a bijection between $A$ and $B$.

Assume such two functions exist. Fix the two functions $\widehat{f} : A \to B$ and $\widehat{g} : B \to A$ be one-to-one functions.

Then there exist bijections $\widehat{f}^{-1} : \widehat{f}(A) \to A$ and $\widehat{g}^{-1} : g(B) \to B$.

Define $A_C \subseteq A$ as the set of such elements $a \in A$ that $(\widehat{g} \circ \widehat{f})^n a = a$ for some integer $n \in \mathbb{N}$ (zero is not in $\mathbb{N}$).

In the same way, define $B_C = \{b \in B \mid \exists n \in \mathbb{N} : (\widehat{f} \circ \widehat{g})^n b = b\}$.

For a given element $a \in A$, define a sequence of ancestors $X(a) = (a, \widehat{g}^{-1}(a), \widehat{f}^{-1}(\widehat{g}^{-1}(a)), \ldots)$. The sequence is finite if the last element is in $A \setminus \widehat{g}(B)$ or in $B \setminus \widehat{f}(A)$, and infinite otherwise. In the same way, for $b \in B$, define $X(b) = (b, \widehat{f}^{-1}(b), g^{-1}(\widehat{f}^{-1}(b)), \ldots)$.

Define $A_I = \{a \in A \mid X(a) \text{ is infinite}\}$, $A_E = \{a \in A \mid |X(a)| \text{ is even}\}$, $A_O = \{a \in A \mid |X(a)| \text{ is odd}\}$. Observe that $A_I, A_E, A_O$ are disjoint and cover $A$. Similarly, define $B_I, B_E, B_O$.

Notice that $A_C \subseteq A_I$, since for any $a \in A_C$, $X(a)$ is periodic and infinite.

I claim that $\widehat{f}$ maps $A_I$ bijectively onto $B_I$. To see this, we need to show that for any $b \in B_I$, there exists a preimage $a \in A_I$ such that $\widehat{f}(a) = b$. Such $a$ is the second element of $X(b)$, $a = \widehat{f}^{-1}(b)$. Then $\widehat{f}(a) = b$. $a$ is indeed in $A_I$, because $X(a) = (a = \widehat{f}^{-1}(b), \widehat{g}^{-1}(\widehat{f}^{-1}(b)), \ldots)$ is also infinite.

Next, show that $f$ maps $A_E$ bijectively onto $B_O$. To see this, we need to show that for any $b \in B_O$, there exists a preimage $a \in A_E$ such that $\widehat{f}(a) = b$. Such $a$ is the second element of $X(b)$, $a = \widehat{f}^{-1}(b)$. Then $\widehat{f}(a) = b$. $a$ is indeed in $A_E$, because $X(a) = (a = \widehat{f}^{-1}(b), \widehat{g}^{-1}(\widehat{f}^{-1}(b)), \ldots)$. Then $|X(a)| = |X(b)| - 1$ is even.

In the same way, $g$ maps $B_E$ bijectively onto $A_O$. Therefore, $g^{-1}$ maps $A_O$ bijectively onto $B_E$.

Define a map $h : A \to B$:

$$h(a) = \begin{cases} \widehat{f}(a), & a \in A_I \\ \widehat{f}(a), & a \in A_E \\ \widehat{g}^{-1}(a), & a \in A_O \end{cases}, \tag{1}$$

which is the desired bijection between $A$ and $B$. □

# 2 Homework 2

**Theorem 2.1** (Knaster-Tarski)**.** *Let $(A, \sqsubseteq)$ be a complete lattice and let $F$ be a monotone function on $A$. Further, let $\hat{y} = \bigsqcup\{x \mid x \sqsubseteq F(x)\}$ and $\hat{z} = \bigsqcap\{x \mid F(x) \sqsubseteq x\}$. It holds, that:*

1. *$\hat{y}$ and $\hat{z}$ are fixpoints of $F$,*
2. *for all fixpoints $x$ of $F$, $\hat{z} \sqsubseteq x \sqsubseteq \hat{y}$*

*Proof.* Suppose $(A, \sqsubseteq)$ is a complete lattice and let $F$ be a monotone function on $A$. Let $U$ be the set of elements $x \in A$ for which $x \sqsubseteq F(x)$ and let $D$ be the set of elements $x \in A$ for which $x \sqsupseteq F(x)$. Then as described in the lecture let us denote the join and meet of these two sets respectively as $\hat{y} = \bigsqcup U$ and $\hat{z} = \bigsqcap D$ (this meet and join exist because the considered lattice is complete). We shall prove:

1. $\hat{y}$ and $\hat{z}$ are fixpoints of $F$.
2. For all fixpoints $x$ of $F$ the following holds $\hat{z} \sqsubseteq x \sqsubseteq \hat{y}$.

**1.** Let us first prove that $\hat{y}$ is a fixpoint of $F$. Let us pick an arbitrary element $u \in U$, then by definition of $\hat{y}$ we have $u \sqsubseteq \hat{y}$. Because $F$ is monotone this implies $F(u) \sqsubseteq F(\hat{y})$. And because $u$ was picked arbitrarily from $U$ this means that $F(\hat{y})$ is an upper bound on $U$. But $\hat{y}$ is the least upper bound of $U$ and thus $\hat{y} \sqsubseteq f(\hat{y})$. Again by applying the monotonicity of $F$ we get $F(\hat{y}) \sqsubseteq F(F(\hat{y}))$ and we can conclude $F(\hat{y}) \in U$. But because $\hat{y}$ is the join of $U$ and $F(\hat{y})$ is an element of $U$ it follows $F(\hat{y}) \sqsubseteq \hat{y}$. Together with the inequality $\hat{y} \sqsubseteq F(\hat{y})$ this gives us $F(\hat{y}) = \hat{y}$, hence $\hat{y}$ is a fixpoint of $F$. The proof that $\hat{z}$ is a fixpoint of $F$ can be done along the same lines by considering $\sqsupseteq$ instead of $\sqsubseteq$ and the set $D$ instead of $U$.

**2.** Suppose $\hat{x}$ is a fixpoint of $F$. Then clearly $\hat{x} \in U$ and hence $\hat{x} \sqsubseteq \hat{y}$ as $\hat{y}$ is the join of $U$. Similarly $\hat{x} \in D$ and hence $\hat{x} \sqsupseteq \hat{z}$ as $\hat{z}$ is the meet of $D$. So it follows that $\hat{z} \sqsubseteq \hat{x} \sqsubseteq \hat{y}$ and the proof is done. □

# 3 Homework 4

## 3.1 Sorting

**Definition 3.1** (`ordered`)**.** A function $\texttt{ordered} : \Sigma^* \to \{true, false\}$ is defined as follows. $\texttt{ordered}(\varepsilon) = true$, $\texttt{ordered}(a) = true$, $\texttt{ordered}(abx) = (a \leq b) \wedge \texttt{sorted}(bx)$, where $a \in \Sigma$, $x \in \Sigma^*$.

**Lemma 3.1.** $\texttt{sorted}(\texttt{ms}(x)) = true$.

*Proof.* We will use induction on the length of $x$. Obviously, $\texttt{sorted}(\texttt{ms}(\varepsilon)) = \texttt{sorted}(\texttt{ms}(a)) = true$.

Recall that $\texttt{ms}(x) = \texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$. By the induction assumption, we know that $\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x))$ are sorted, since their length is less than the length of $x$. Now it is enough to show that $\texttt{sorted}(\texttt{merge}(y, z)) = true$ if $y$ and $z$ are sorted. Consider $\texttt{sorted}(\texttt{merge}(ay, bz))$. Without loss of generality, let $a \leq b$, so $\texttt{sorted}(\texttt{merge}(ay, bz)) = \texttt{sorted}(a\texttt{merge}(y, bz)) = true$, since $a$ is less or equal to both $b$ and the next element of $y$, and $\texttt{sorted}(\texttt{merge}(y, bz)) = true$ holds by induction. □

**Definition 3.2** (`permutation`)**.** For all natural $n$, the function $\texttt{permutation} : \Sigma^n \times \Sigma^n \to \{true, false\}$ is defined as follows. $\texttt{permutation}(\varepsilon, \varepsilon) = true$, $\texttt{permutation}(a, b) = (a == b)$, $\texttt{permutation}(ax, yaz) = \texttt{permutation}(x, yz)$, and if $a \notin y$ then $\texttt{permutation}(ax, y) = false$, where $a, b \in \Sigma$, $x, y, z \in \Sigma^*$.

**Lemma 3.2.** $\texttt{permutation}(x, \texttt{ms}(x)) = true$.

*Proof.* We will use induction on the length of $x$. Obviously, $\texttt{permutation}(\varepsilon, \texttt{ms}(\varepsilon)) = \texttt{permutation}(a, \texttt{ms}(a)) = true$.

First, let's show that $\texttt{merge}(\texttt{x}, \texttt{y})$ is a permutation of $xy$. Indeed, if, for instance, $a \leq b$, then $\texttt{permutation}(\texttt{merge}(ax, by), axby) = \texttt{permutation}(a\texttt{merge}(x, by), axby) = \texttt{permutation}(\texttt{merge}(x, by), xby) = true$ by induction.

Finally, $\texttt{ms}(x) = \texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$. By the induction assumption, we know that $\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x))$ are permutations of $\texttt{odd}(x), \texttt{even}(x)$ respectively, since their length is less than the length of $x$. Moreover, as we have already shown above, $\texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$ is a permutation of $\texttt{ms}(\texttt{odd}(x))\texttt{ms}(\texttt{even}(x))$, which is a permutation of $\texttt{odd}(x)\texttt{even}(x)$. Finally, it is sufficient to prove that $\texttt{odd}(x)\texttt{even}(x)$ is a permutation of $x$. Using induction, $\texttt{permutation}(\texttt{odd}(ax)\texttt{even}(ax), ax) = \texttt{permutation}(a\texttt{even}(x)\texttt{odd}(x), ax) = \texttt{permutation}(\texttt{even}(x)\texttt{odd}(x), x) = \texttt{permutation}(\texttt{odd}(x)\texttt{even}(x), x) = true$. Here, we used the induction hypothesis and the fact that $xy$ is a permutation of $yx$ (also straightforwardly proved with induction, if needed). □

## 3.2 The human and monkeys theorem

Let's have a relation $<$, defined as follows: $x < y$ if $\texttt{parent}(x, y)$ or there exists $z$ such that $x < z$ and $\texttt{parent}(z, y)$. We will assume that this relation is well-founded. Also, for all $x$, either $\texttt{human}(x)$ or $\texttt{monkey}(x)$.

**Theorem 3.3.** *If $x < y$, and* $\texttt{human}(y)$*, and* $\texttt{monkey}(x)$*, then there exist $z_1, z_2$ such that* $\texttt{parent}(z_1, z_2)$*, and* $\texttt{human}(z_2)$*, and* $\texttt{monkey}(z_1)$*.*

*Proof.* Define a sequence $w_1, w_2, \ldots$ inductively like this: $w_1 = y$, $w_{n+1}$ is such that $\texttt{parent}(w_{n+1}, w_n)$, and $x < w_{n+1}$. If $w_n = x$, then terminate the sequence. Obviously, $w_1 > w_2 > \ldots$. Since $<$ is well-founded, the sequence has to be finite: $y = w_1 > w_2 > \ldots > w_k = x$.

For the sake of contradiction, suppose, there is no such $i$, that $\texttt{parent}(w_{i+1}, w_i)$, and $\texttt{human}(w_i)$, and $\texttt{monkey}(w_{i+1})$. Then, by induction, all $w_i$ are humans. This contradicts the fact that $w_k$ is a monkey. Therefore, the desired $z_1, z_2$ exist among some $w_{i+1}, w_i$, as requested. □

# 4 Homework 6: Soundness of FOL Resolution

In the following we use sets for both representing formulas in CNF and individual clauses. Hence, a formula in CNF is a set containing sets of literals.

We prove the soundness of the following FOL resolution rule:

$$\frac{C_1[l_1] \qquad C_2[\neg l_2]}{C_1\theta[\bot] \vee C_2\theta[\bot]} \; l_1\theta = l_2\theta$$

**Lemma 4.1.** *For any $C_1[l]$, $C_2[\neg l]$ first-order clauses and $\mathcal{I}$ interpretation, we get*

$$\mathcal{I} \models \{C_1[l], C_2[\neg l]\} \Rightarrow \mathcal{I} \models C_1[\bot] \vee C_2[\bot].$$

*Proof.* Let $\mathcal{I}$, $C_1[l]$, $C_2[\neg l]$ be arbitrary and $\mathcal{I} \models C_1[l]$ and $\mathcal{I} \models C_2[\neg l]$. We show $C_1[\bot] \vee C_2[\bot]$. Consider the two cases for $[\![l]\!]_{\mathcal{I}}$:

**Case 1.** $[\![l]\!]_{\mathcal{I}} = true$ **:** Consider the clause $C_2' = C_2 \setminus \{\neg l\}$, corresponding to the clause $C_2$ with $\neg l$ removed. Since $[\![\neg l]\!]_{\mathcal{I}} = false$, and $\mathcal{I} \models C_2[\neg l]$, $\mathcal{I} \models C_2'$. Moreover, since $\mathcal{I} \models C_2'$, $\mathcal{I} \models C_2' \cup \{\bot\}$ and therefore $\mathcal{I} \models C_2[\bot]$. Finally, since $\mathcal{I} \models C_2[\bot]$, $\mathcal{I} \models C_1[\bot] \vee C_2[\bot]$.

**Case 2.** $[\![l]\!]_{\mathcal{I}} = false$ **:** symmetric to Case 1. □

**Lemma 4.2.** *Let $\mathcal{I}$, $C$ be an arbitrary interpretation and clause, and $\sigma$ a substitution.*

$$\mathcal{I} \models \forall x_1 ... x_n, C \Rightarrow \mathcal{I} \models \forall x_1 ... x_n, C\sigma$$

*Proof.* Let $\mathcal{I}, C, \sigma$ be arbitrary with $\mathcal{I} \models \forall x_1 ... x_n C$, and assume towards a contradiction that $\mathcal{I} \not\models \forall x_1 ... x_n, C\sigma$. By the definition of $[\![\forall x.\phi]\!]$, there must be a mapping $\kappa = [y_1 \to d_1, ..., y_n \to d_n]$ from free variables in $C\sigma$ to $d_1 ... d_n \in D_{\mathcal{I}}$ s.t. $[\![C\sigma]\!]_{\mathcal{I}[\kappa]} = \text{false}$.
We now construct a mapping $\kappa' = x_1 \to d_1', ..., x_n \to d_n'$ s.t. $[\![C]\!]_{\mathcal{I}[\kappa']} = false$. Consider the syntactic substitution $\sigma = x_1 \to t_1, ..., x_n \to t_n$, we simply define $\kappa' = [x_1 \to t_1\kappa, ..., x_n \to t_n\kappa]$. Hence, $\kappa'$ is a map from free variables in $C$ to elements in the domain $D_{\mathcal{I}}$ and is a valid mapping to close the formula $C$.

Moreover, $[\![C]\!]_{\mathcal{I}[\kappa']} = [\![C\sigma]\!]_{\mathcal{I}[\kappa]} = false$. Hence, $\mathcal{I} \not\models \forall x_1, ..., x_n C$, a contradiction. □

**Theorem 4.3** (Soundness of FOL resolution)**.**
*For any clause set $S$, clauses $C_1[l_1] \in S$, $C_2[\neg l_2] \in S$, and substitution $\theta$ s.t. $l_1\theta = l_2\theta$: $S$ is satisfiable iff $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ is satisfiable.*

*Proof.*

$\Rightarrow$ Let $S$, $C_1[l_1] \in S$, $C_2[\neg l_2] \in S$ s.t. $l_1\theta = l_2\theta$, be arbitrary and $S$ satisfiable. By definition of satisfiable, let $\mathcal{I}$ s.t. $\mathcal{I} \models S$ and therefore $\mathcal{I} \models C_1$, and $\mathcal{I} \models C_2$. Note, that both clauses $C_1$ and $C_2$ are implicitely universally quantified, and since $\forall$ distributes over $\wedge$, we get $\mathcal{I} \models \forall x_1...x_n, C_1$ and $\mathcal{I} \models \forall x_1...x_n, C_2$ where $x_1...x_n$ are the free variables in $S$.
By Lemma 4.2, $\mathcal{I} \models \forall x_1...x_n, C_1\theta$ and $\mathcal{I} \models \forall x_1...x_n, C_2\theta$. In the following we drop the implicitely quantified variables again. Let $l = l_1\theta = l_2\theta$ and since the order of substitution is irrelevant, $\mathcal{I} \models C_1[l]\theta$ and $\mathcal{I} \models C_2[l]\theta$. By Lemma 4.1, $\mathcal{I} \models C_1\theta[\bot] \vee C_2\theta[\bot]$, and therefore, by definition of satisfiable, $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ is satisfiable.

$\Leftarrow$ Let $S$, $C_1[l_1] \in S$, $C_2[\neg l_2] \in S$ s.t. $l_1\theta = l_2\theta$, be arbitrary and $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ satisifable. By definition of satisfiable, let $\mathcal{I}$ s.t. $\mathcal{I} \models S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$. By definition of $\wedge$, $\mathcal{I} \models S$ and therefore $S$ is satisfiable. □

# 5 Homework 8: Congruence Closure Algorithm and Complexity

In this assignment, we will demonstrate that the runtime of the congruence closure implemented using a *union-find* data structure is in $\mathcal{O}(n \log n)$. To this end, we will first introduce the union-find data structure and its operations and then give a precise description of the congruence closure algorithm, analyzing each step in terms of its complexity.

## 5.1 Union-Find Data Structure

The union-find data structure, also known as the disjoint-set data structure, efficiently maintains a collection of disjoint sets. Each set is represented as a tree, with each node pointing to its parent. The root of each tree is the so-called *representative* of that set.

The union-find data structure supports two main operations:

- `find`$(x)$: Determines the representative of the set containing the element $x$.
- `union`$(x, y)$: Merges the sets with representatives $x$ and $y$ into a single set.

### 5.1.1 Optimizations

To improve the efficiency of union-find operations, one can employ two standard heuristics:

- **Path Compression**: During a `find` operation, we make each node on the search path point directly to the root. This flattens the structure of the tree, leading to faster subsequent `find` operations.
- **Union by Rank (or Size)**: Let the *rank* of a node in the union find structure denote an upper bound on the height of the subtree rooted at that node. When performing a `union` operation, we attach the tree with the smaller rank (or size) to the root of the tree with the larger rank. This prevents the trees from becoming too tall.

### 5.1.2 Time Complexity

With these optimizations, both the `find` and `union` operations have an amortized time complexity of $\mathcal{O}(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function. The function $\alpha(n)$ grows extremely slowly and can be considered a constant (less than 5) for all practical values of $n$.

As this exercise assumes that the union-find data structure is available, we will not give a detailed proof of the time complexity of the operations.

## 5.2 Congruence Closure Algorithm

The congruence closure algorithm computes the smallest congruence relation over terms induced by a given set of equations. This is fundamental in automated reasoning and term rewriting systems.

We first give a formal description of the problem:

- **Signature** $\Sigma$: A set of function symbols $F$, and predicate symbols $P$, each with an associated arity.
- **Terms** $T$: The set of all terms constructed from $\Sigma$ and a set of variables, using the usual rules of term formation.
- **Equations** $E$: A finite set of equations $s_i = t_i \mid 1 \leq i \leq m$ between terms in $T$.
- **Congruence Relation**: An equivalence relation that is compatible with the function and predicate symbols in $\Sigma$; that is, if $s_i \equiv t_i$ for $1 \leq i \leq k$, then $f(s_1, \ldots, s_k) \equiv f(t_1, \ldots, t_k)$ for any $f \in F$ and $p(s_1, \ldots, s_k) \equiv p(t_1, \ldots, t_k)$ for any $p \in P$.

The goal of the procedure is to compute the smallest congruence relation $\equiv$ over $\mathcal{T}$ such that all equations in $E$ are satisfied: $s_i \equiv t_i$ for all $1 \leq i \leq m$.

The algorithm proceeds in three main phases: Initialization, Processing Input Equations, and Enforcing Congruence Closure. We will describe each phase and analyze its time complexity.

**Initialization**

- For each term $t \in T$, create a singleton set $t$ in the union-find data structure.
- Initialize an empty **signature table** $\mathcal{S}$, typically implemented as a hash table, to map signatures to representative terms.

**Time Complexity** $\mathcal{O}(n)$, where $n = |T|$.

**Processing Input Equations**

For each equation $s = t$ in $E$:

- Perform $\texttt{union}(\texttt{find}(s), \texttt{find}(t))$ to merge the sets containing $s$ and $t$.

**Time Complexity** $\mathcal{O}(m \cdot \alpha(n))$, where $m = |E|$.

**Enforcing Congruence Closure**

- Repeat until no new unions are performed:
  - For each compound term $f(t_1, t_2, \ldots, t_k) \in T$:
    - Let $r_i = \texttt{find}(t_i)$ for $1 \leq i \leq k$.
    - Construct the *signature* $\sigma = (f, r_1, r_2, \ldots, r_k)$.
    - If $\sigma$ is in $\mathcal{S}$ with associated term $s$:
      - Perform $\texttt{union}(\texttt{find}(f(t_1, \ldots, t_k))), \texttt{find}(s))$.
    - Else:
      - Add $\sigma$ to $\mathcal{S}$ with associated term $f(t_1, \ldots, t_k)$.

**Time Complexity** We need to consider the needed number of iterations and the time complexity per iterations.

For the time complexity per iteration, finding the representatives for each compound term is in $\mathcal{O}(k \cdot \alpha(n))$, where $k$ is the maximum arity of functions and predicates in $\Sigma$. As we consider constant-time hash table operations, constructing the signature and looking it up in $\mathcal{S}$ is in $\mathcal{O}(1)$. The union operation takes $\mathcal{O}(\alpha(n))$ time. Thus, the total time per iteration is in $\mathcal{O}(k \cdot \alpha(n))$.

For the number of iterations, observe that each `union` operation reduces the number of distinct equivalence classes. Since there are at most $n$ terms, there can be at most $n - 1$ unions. However, the structure of the terms may cause the same unions to be considered multiple times. We claim that the number of iterations is bounded by $\mathcal{O}(\log n)$.

We claim that, with the mentioned optimizations, the maximum rank $R_{\max}$ in a union-find data structure with $n$ elements is $\lfloor \log_2 n \rfloor$, which we will show using the following Lemma:

**Lemma 5.1.** *In the union-find data structure with union by rank, the minimum number of nodes in a tree of rank $r$ is $2^r$.*

*Proof.* We prove this by an inductive argument, considering that a tree's rank can only increase due to `union` operations.

*Base case*: For $r = 0$, the tree is a singleton node, and the minimum number of nodes is $1 = 2^0$.

*Inductive step*: Let the rank of a tree be the maximum rank of any node in that tree. When two trees of rank $r_1$ and $r_2$ are united and $r_1 \neq r_2$, the resulting tree has rank $\max(r_1, r_2)$ due to the union by rank heuristic and thus contains at least $2^{\max(r_1,r_2)}$ nodes.

Whenever two trees of the same rank $r$ are united, the resulting tree has rank $r + 1$, as one tree is attached to the other ones root. Thus, the resulting tree has at least $2^r + 2^r = 2^{r+1}$ nodes by the induction hypothesis. □

Observe that path compression will only reduce the height of a node's subtree, and thus has no effect on the presented lower bound.

From Lemma 5.1, we immediately obtain the following corollary:

**Corollary 5.2.** *The maximum rank $R_{\max}$ of any node in the union-find data structure with union by rank is at most $\lfloor \log_2 n \rfloor$.*

**Lemma 5.3.** *The maximum number of times the representative (root) of any term can change due to `union` operations is bounded by $R_{\max}$.*

*Proof.* Each time a `union` operation changes the parent of a root node, it is because two trees are being merged (non-root nodes are unaffected by `union`). The rank of a root node increases only when two roots of equal rank are united, and as the maximum rank that can occur is $R_{\max}$, the `union` operation can change the representative of any term at most $R_{\max}$ times. □

According to Lemma 5.3, each term can involved in at most $R_{\max}$ union operations that change its rank. Therefore, all terms have stabilized with respect to their parent nodes after at most $R_{\max} \in \mathcal{O}(\log n)$ iterations.

## 5.3 Complexity Analysis

Combining the complexities of the three phases of the algorithm, we obtain the overall time complexity:

$$\mathcal{O}(n) + \mathcal{O}(m \cdot \alpha(n)) + \mathcal{O}(n \cdot k \cdot \alpha(n) \log n) = \mathcal{O}(n \cdot k \cdot \alpha(n) \log n)$$

Observe that the term $k$ can be considered constant for a fixed signature $\Sigma$. As the inverse Ackermann function $\alpha(n)$ grows extremely slowly with $n$ it can be considered a constant for all practical purposes. This gives us the final result of a runtime of $\mathcal{O}(n \log n)$ for the entire algorithm.

# 6 Homework 10: Untyped Lambda Calculus

This homework contains two exercises on the untyped lambda calculus. For readability, we will use the following definitions from the lecture:

$$\begin{array}{lll} \mathrm{I} = \lambda x.x & \top = \lambda xy.x & \bot = \lambda xy.y \\ \mathrm{if} = \lambda bxy.bxy & \underline{0} = \lambda Sx.x & \underline{1} = \lambda Sx.Sx \\ \underline{2} = \lambda Sx.S(Sx) & \underline{n} = \lambda Sx.S^n(x) & \mathrm{add} = \lambda mn.mSn \\ \mathrm{mult} = \lambda mn.m(\mathrm{add}\, n)\,\underline{0} & \mathrm{iszero} = \lambda n.n(\lambda x.\bot)\top & Y = \lambda f.(\lambda x.fxx)(\lambda x.fxx) \end{array}$$

## 6.1 Predecessor function for Church numerals

$$\mathrm{pred} = \lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u)$$

The predecessor function pred should take a numeral $n$ and return $n-1$. We know that:

- $\mathrm{pred}(\underline{0}) = \underline{0}$, since the predecessor of zero should not go below zero.
- For any $n > 0$, $\mathrm{pred}(n) = n - 1$.

A clever technique to define pred uses a "pairing" function that keeps track of two values as we iterate through the numeral. The idea is to transform the iteration function of the numeral into one that carries along a state representing the current and previous "count". By the time we finish applying the numeral's function the appropriate number of times, we end up with the predecessor.

We want an iteration that, given a function $f$ and an initial value $x$, produces something related to $(n-1)$. Consider we apply $n$ times a function that takes a pair $(g, h)$ and returns a new pair that effectively shifts $f$ from "next" to "previous":

1. Start with a pair:
   - A "dummy" function $\lambda u.x$ that, if applied, yields $x$.
   - The identity function $\mathrm{I} = \lambda u.u$ which essentially acts as the successor chain.

   We think of this initial pair as $(\lambda u.x, \lambda u.u)$. The idea is that the second element of the pair eventually ends up "one step ahead" of the first. After iterating $n$ times, the first component of the pair will encode the predecessor computation.

2. Each iteration step transforms $(g, h)$ into $(h, \lambda z.h(gf))$ or something equivalent. More concretely, we define a function $\lambda gh.h(gf)$ that, given a pair $(g, h)$, produces a new function related to this shifting. By repeatedly applying this, we "shift" through the functions, so that after $n$ steps, the "front" shifts out one step ahead, and what remains in the first component is effectively the predecessor function applied to $f$ and $x$.

This yields the following lambda term for pred:

$$\text{pred} = \lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u).$$

We illustrate the correctness of this definition by $\beta$-reducing $\text{pred}\,\underline{0}$ and $\text{pred}\,\underline{2}$:

**Reduction of $\text{pred}\,\underline{0}$**

$$\begin{aligned}
\text{pred}\,\underline{0} &= (\lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u))\,\lambda Sx'.x' \\
&\to_\beta \lambda fx.(\lambda Sx'.x')(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda x'.x')(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.x \\
&= \underline{0}
\end{aligned}$$

**Reduction of $\text{pred}\,\underline{2}$**

$$\begin{aligned}
\text{pred}\,\underline{2} &= (\lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u))\,(\lambda Sx.S(Sx)) \\
&\to_\beta \lambda fx.(\lambda Sx'.S(Sx'))(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda x'.(\lambda gh.h(gf))((\lambda gh.h(gf))x'))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda gh.h(gf))((\lambda gh.h(gf))(\lambda u.x))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda gh.h(gf))(\lambda h.h(x))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda h.h((\lambda h.h(x))f))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda h.h(f(x)))(\lambda u.u) \\
&\to_\beta \lambda fx.f(x) \\
&= \underline{1}
\end{aligned}$$

## 6.2 $\beta$-Reduction of fact 2

For the sake of readability, we assume that all arithmetic functions defined in the lecture correspond to their intended behavior (i.e., we assume that add $nm = n + m$, mult $nm = n \cdot m$, and iszero $n = \top$ if $n = 0$ and $\bot$ otherwise). Steps following from these definitions are written using a double arrow $\Rightarrow_\beta$.

$$
\begin{aligned}
\text{fact}\,\underline{2} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{2} \\
&= ((\lambda f.f(\lambda x.fxx)(\lambda x.fxx))(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{2} \\
&\to_\beta (\lambda x.(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n))))(xx)) \\
&\quad (\lambda x.(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n))))(xx))\,\underline{2} \\
&\to_\beta (\lambda n.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n((Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))(\text{pred}\,n))))\,\underline{2} \\
&\to_\beta \text{if}(\text{iszero}\,\underline{2})\underline{1}(\text{mult}\,\underline{2}(\text{fact}(\text{pred}\,\underline{2}))) \\
&\to_\beta \text{if}(\bot)\underline{1}(\text{mult}\,\underline{2}(\text{fact}\,\underline{1})) \\
&\to_\beta \text{mult}\,\underline{2}(\text{fact}\,\underline{1})
\end{aligned}
$$

The first two reductions of computing fact $\underline{1}$ and fact $\underline{0}$ are analogous to the previous computation and will thus be omitted.

$$
\begin{aligned}
\text{fact}\,\underline{1} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{1} \\
&\Rightarrow_\beta \text{if}(\text{iszero}\,\underline{1})\underline{1}(\text{mult}\,\underline{1}(\text{fact}(\text{pred}\,\underline{1}))) \\
&\to_\beta \text{if}(\bot)\underline{1}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) \\
&\to_\beta (\bot)\underline{1}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) \\
&\to_\beta \text{mult}\,\underline{1}(\text{fact}\,\underline{0})
\end{aligned}
$$

$$
\begin{aligned}
\text{fact}\,\underline{0} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{0} \\
&\Rightarrow_\beta \text{if}(\text{iszero}\,\underline{0})\underline{1}(\text{mult}\,\underline{0}(\text{fact}(\text{pred}\,\underline{0}))) \\
&\to_\beta \text{if}(\top)\underline{1}(\text{mult}\,\underline{0}(\text{fact}\,\underline{0})) \\
&\to_\beta (\top)\underline{1}(\text{mult}\,\underline{0}(\text{fact}\,\underline{0})) \\
&\to_\beta \underline{1}
\end{aligned}
$$

From these computations, we can conclude that $\text{fact}\,\underline{2} = \text{mult}\,\underline{2}(\text{fact}\,\underline{1}) = \text{mult}\,\underline{2}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) = \text{mult}\,\underline{2}(\text{mult}\,\underline{1}\,\underline{1}) = \text{mult}\,\underline{2}\,\underline{1} = \underline{2}$.